

MARCO FERNSTAEDT

FULL-STACK JAVASCRIPT ENGINEER | APPLICATION MODERNIZATION

Phoenix, Arizona | fernstaedtmarco@gmail.com | github.com/MarcoFernstaedt | marcofernstaedt.com

PROFESSIONAL SUMMARY

Integrated Systems Engineer who turns operational pain points into secure, testable software. Experience modernizing workflows with TypeScript, React, Next.js, Node.js, REST APIs, C#.NET, relational data, automated testing, and CI/CD. Strong at extending existing systems incrementally, mapping legacy behavior, protecting working functionality, and shipping accessible operator experiences.

TECHNICAL SKILLS

JavaScript / TypeScript: JavaScript (ES6+), TypeScript, React, Next.js, Node.js, Fastify, Express, Vite

Backend / Data: REST APIs, ASP.NET Core, C#.NET, SQLite, MongoDB, schema migrations, authentication, RBAC

Quality / Delivery: Vitest, Playwright, Testing Library, xUnit, integration testing, GitHub Actions, Git, npm, CI/CD

Platforms / Practices: Windows services, WPF, Linux/VPS, Vercel, PWA, accessibility, secure design, incremental modernization

PROFESSIONAL EXPERIENCE

Mashura LLC | Integrated Systems Engineer | Phoenix, Arizona | July 2025 to Present

- Originated the idea for an internal cabinet fleet-support platform after identifying recurring support and remote-diagnostic bottlenecks, then developed the first secure end-to-end software vertical slice in C#.NET.
- Modernized an existing platform instead of replacing it: extended a WPF support console, ASP.NET Core API, Windows service agent, SQLite persistence, and typed command pipeline while preserving established behavior.
- Delivered a tested private lab release with outbound HTTPS connectivity, unique device enrollment, Supervisor approval, client/site/cabinet organization, role-based staff access, heartbeats, process and event diagnostics, and connectivity incident workflows.
- Designed the platform toward an approximately 8,000-cabinet fleet model using desired-versus-actual state, versioned policy, narrow allowlisted actions, audit history, replay protection, bounded payloads, and rollback-oriented remediation patterns.
- Built automated coverage for enrollment, authentication, authorization, connectivity, stale and lost responses, firewall-result validation, and audit behavior; the focused shipping suite passed more than 1,900 tests.
- Translated frontline support experience into technical requirements and operator workflows, connecting customer-impacting incidents to practical diagnostics, safer remediation, and future measurable reductions in remote sessions and resolution time.

SELECTED FULL-STACK JAVASCRIPT PROJECTS

Household Coordinator PWA | TypeScript, React, Fastify, Node.js, SQLite, Vite

- Built a production-minded, installable PWA with server-enforced tenant isolation, owner/member authorization, expiring scoped guest access, optimistic concurrency, replay-safe mutations, and truthful offline conflict recovery.
- Implemented schema-validated APIs, Argon2id authentication, HttpOnly sessions, CSRF/origin checks, rate limiting, migrations, and automated unit, accessibility, browser, and offline tests with Vitest and Playwright.

Accessible Image-to-Audio Platform | Next.js, React, TypeScript, REST APIs, Vercel

- Developed and deployed a full-stack application that validates uploaded images, generates descriptions and spoken audio through server-side APIs, and returns binary audio for accessible playback and download.
- Added keyboard and screen-reader workflows, semantic status reporting, server-side validation, rate limiting, bot protection, secret isolation, CI checks, and production-build verification.

Software Engineering Portfolio and Publishing Workflow | Next.js, React, TypeScript, GitHub Actions

- Built an accessible responsive portfolio with typed content, server-rendered metadata, API-backed GitHub activity, deterministic release checks, and an approval-gated publishing workflow that prevents accidental publication.

ENGINEERING APPROACH

Incremental modernization | Legacy-system discovery | API and data-model design | Secure-by-default implementation | Test-driven delivery | Accessible UI engineering | Operational documentation | Cross-functional problem solving